

Commonly used Design Patterns in .NET Core and microservices applications:

🌱 What is a Design Pattern?

A **Design Pattern** is a proven, reusable solution to a common software design problem.

It's **not** code, but a **template** or **guideline** for how to solve a particular type of problem in a clean, maintainable, and flexible way.

Think of it like **best practices** or **recipes** that experienced developers follow to handle recurring challenges in software design.

🏠 Real-world Analogy (Easy to Remember)

Analogy 1: House Construction

When you build a house, you don't start from scratch every time — you use **standard designs**:

- Bedrooms usually have doors and windows.
- Kitchens have sinks, cabinets, and gas connections.

Similarly, in software development:

- You don't reinvent how to create objects or manage dependencies.
- You use **design patterns** like **Factory**, **Singleton**, or **Observer** to structure your code efficiently.

💡 *"Just like architects reuse building blueprints, software engineers reuse design patterns."*

💬 In Technical Terms

Design Patterns help you achieve:

- **Loose coupling** between components
- **High reusability** of code
- **Ease of maintenance and scalability**
- **Better communication** among developers using a common vocabulary (e.g., "use Factory pattern here")

⚙️ Types of Design Patterns

There are 3 main categories:

Category	Purpose	Example Patterns
Creational	Object creation logic	Singleton, Factory, Abstract Factory, Builder, Prototype
Structural	Composition of classes/objects	Adapter, Decorator, Facade, Proxy, Composite
Behavioral	Object interaction and communication	Observer, Strategy, Command, Mediator, State

"A design pattern is a reusable solution to a common software design problem.

It's like a proven recipe or blueprint that helps developers solve design challenges efficiently.

For example, just as architects reuse building blueprints to design houses faster, developers use design patterns like Factory or Singleton to structure software in a standardized, maintainable way."

1) Dependency Injection – A Creational Design Pattern:

🌿 Dependency Injection (DI) – Creational Design Pattern

💡 Definition

Dependency Injection (DI) is a creational design pattern that helps remove **hard-coded dependencies** between classes, promoting **loose coupling**, **testability**, and **flexibility**.

Instead of a class creating its own dependencies, those dependencies are **provided (injected)** from the outside — often by a framework or **IoC (Inversion of Control)** container such as the one built into **.NET Core**.

💬 Simple Explanation

Without DI, a class decides how to create or manage the objects it depends on.

With DI, the class only focuses on **what it needs to do**, while another component (the container or calling code) provides the required dependencies.

🏠 Real-World Analogy

Imagine living in a rented apartment:

- You use **electricity**, **gas**, and **internet**.
- You don't generate your own electricity or run your own internet — these services are **provided by external suppliers**.

You simply **plug in your device and use it** — regardless of how the electricity is produced.

That's **Dependency Injection** — you depend on an **abstraction (socket)**, not the **actual implementation (power plant)**.

✗ Bad Example (Anti-Pattern) — Tight Coupling

csharp

 Copy code

```
public interface IMessageService
{
    void Send(string message);
}

public class EmailService : IMessageService
{
    public void Send(string message)
    {
        Console.WriteLine($"Email sent: {message}");
    }
}

public class NotificationManager
{
    private EmailService _emailService;

    public NotificationManager()
    {
        // ✗ Creating dependency inside the class
        _emailService = new EmailService();
    }

    public void Notify(string message)
    {
        _emailService.Send(message);
    }
}

class Program
{
    static void Main()
    {
        NotificationManager manager = new NotificationManager();
        manager.Notify("Hello World!");
    }
}
```

Output

yaml

 Copy code

```
Email sent: Hello World!
```

⚠ Problems

- `NotificationManager` is tightly coupled to `EmailService`.
- Cannot easily replace `EmailService` with another implementation (like SMS).
- Hard to test, as dependencies are created internally.

✓ Good Example (Using Dependency Injection)

```
csharp Copy code

public interface IMessageService
{
    void Send(string message);
}

public class EmailService : IMessageService
{
    public void Send(string message)
    {
        Console.WriteLine($"Email sent: {message}");
    }
}

public class SmsService : IMessageService
{
    public void Send(string message)
    {
        Console.WriteLine($"SMS sent: {message}");
    }
}

public class NotificationManager
{
    private readonly IMessageService _messageService;

    // ✓ Dependency is injected via constructor
    public NotificationManager(IMessageService messageService)
    {
        _messageService = messageService;
    }

    public void Notify(string message)
    {
        _messageService.Send(message);
    }
}
```

```
class Program
{
    static void Main()
    {
        // Inject EmailService
        IMessageService emailService = new EmailService();
        NotificationManager emailManager = new NotificationManager(emailService);
        emailManager.Notify("Email notification sent");

        // Inject SmsService
        IMessageService smsService = new SmsService();
        NotificationManager smsManager = new NotificationManager(smsService);
        smsManager.Notify("SMS notification sent");
    }
}
```

Output

yaml

 Copy code

```
Email sent: Email notification sent
SMS sent: SMS notification sent
```

In .NET Core

In a real .NET Core project, dependencies are registered and resolved automatically by the **built-in IoC container**:

csharp

 Copy code

```
services.AddScoped<IMessageService, EmailService>();
```

When `NotificationManager` is requested, the container automatically provides an instance of `EmailService`.

Advantages

Advantage	Description
Loose Coupling	Classes depend on interfaces, not concrete implementations.
Easy Unit Testing	Dependencies can be replaced with mock or fake objects.
High Flexibility	Swap implementations without changing dependent classes.
Better Maintainability	Changes are localized and isolated.
Supports SOLID Principles	Especially the <i>Dependency Inversion Principle (DIP)</i> .

Disadvantages

Disadvantage	Description
Initial Complexity	Can be confusing for beginners.
Slight Performance Overhead	Object resolution by IoC container adds a small cost.
Hidden Dependencies	Improper registration can make debugging harder.
Overuse Risk	Applying DI to small apps can lead to unnecessary complexity.

Alternatives / Related Patterns

Alternative Pattern	Description
Service Locator	Central registry that provides dependencies (less transparent, not recommended).
Factory Pattern	Used to create instances manually when DI container isn't available.
Manual Injection	Dependencies are passed directly without an IoC container.

Testing Example (Using Fake Implementation)

Testing becomes much easier since you can inject fake or mock dependencies.

```
csharp Copy code

public class FakeMessageService : IMessageService
{
    public void Send(string message)
    {
        Console.WriteLine($"[TEST] Message sent: {message}");
    }
}

class Test
{
    static void Main()
    {
        IMessageService fakeService = new FakeMessageService();
        NotificationManager testManager = new NotificationManager(fakeService);
        testManager.Notify("Testing Dependency Injection");
    }
}
```

Output

```
csharp Copy code

[TEST] Message sent: Testing Dependency Injection
```

Summary

Dependency Injection (DI) is a **creational pattern** that promotes loose coupling by providing dependencies from outside a class rather than creating them internally. It improves maintainability, testability, and flexibility — making it a cornerstone of modern .NET Core and microservice architectures.

2)Factory – A Creational Design Pattern:

✿ Factory Design Pattern – [Creational Pattern]

💡 Definition

The **Factory Pattern** is a **creational design pattern** that provides an interface for creating objects **without exposing the creation logic** to the client.

Instead of using the `new` keyword directly, the object creation responsibility is **delegated to a factory class** or method.

In short, the **Factory pattern** **hides the instantiation logic** and **decides which concrete class to instantiate at runtime**.

🧠 Simple Explanation

Instead of a client (caller) creating objects manually, the factory pattern lets a dedicated class handle the creation process.

This helps when you have multiple related classes and want to **decouple object creation from business logic**.

🏠 Real-World Analogy

Imagine a **Car Manufacturing Factory**:

- You (the client) want a car, but you don't care how it's built.
- You just specify the **type** of car — "SUV", "Sedan", or "Hatchback".
- The **factory** takes your request and delivers the correct car.

You don't have to know which parts to use or how to assemble them — the factory hides that complexity.

👉 Similarly, in code, a **Factory class** decides **which concrete class to instantiate** based on the input.

✗ Bad Example (Anti-Pattern) — Tight Coupling

Here, the client directly decides which class to instantiate.

```
csharp Copy code

public interface IVehicle
{
    void Drive();
}

public class Car : IVehicle
{
    public void Drive()
    {
        Console.WriteLine("Driving a Car...");
    }
}

public class Bike : IVehicle
{
    public void Drive()
    {
        Console.WriteLine("Riding a Bike...");
    }
}

class Program
{
    static void Main()
    {
        string vehicleType = "Car";

        // ✗ Direct instantiation - tightly coupled to specific class
        IVehicle vehicle;

        if (vehicleType == "Car")
            vehicle = new Car();
        else
            vehicle = new Bike();

        vehicle.Drive();
    }
}
```

Output

```
csharp Copy code

Driving a Car...
```

⚠ Problems

- The client code (`Main`) decides which class to create.
- If new vehicle types (e.g., Truck, Scooter) are added, you must **modify** the client code.
- **Violates Open/Closed Principle (OCP)** — code should be open for extension, but closed for modification.

✓ Good Example (Using Factory Pattern)

csharp

Copy code

```
public interface IVehicle
{
    void Drive();
}

public class Car : IVehicle
{
    public void Drive()
    {
        Console.WriteLine("Driving a Car...");
    }
}

public class Bike : IVehicle
{
    public void Drive()
    {
        Console.WriteLine("Riding a Bike...");
    }
}

// ✓ Factory class encapsulates object creation logic
public static class VehicleFactory
{
    public static IVehicle GetVehicle(string vehicleType)
    {
        return vehicleType.ToLower() switch
        {
            "car" => new Car(),
            "bike" => new Bike(),
            _ => throw new ArgumentException("Invalid vehicle type")
        };
    }
}
```

```
class Program
{
    static void Main()
    {
        // Factory hides the creation logic
        IVehicle vehicle1 = VehicleFactory.GetVehicle("Car");
        vehicle1.Drive();

        IVehicle vehicle2 = VehicleFactory.GetVehicle("Bike");
        vehicle2.Drive();
    }
}
```

Output

csharp

Copy code

```
Driving a Car...
Riding a Bike...
```

How to Test in Main Method

You can test various scenarios easily without changing client logic:

```
csharp Copy code

class Program
{
    static void Main()
    {
        try
        {
            IVehicle car = VehicleFactory.GetVehicle("Car");
            car.Drive();

            IVehicle bike = VehicleFactory.GetVehicle("Bike");
            bike.Drive();

            // Testing with invalid type
            IVehicle truck = VehicleFactory.GetVehicle("Truck");
            truck.Drive();
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error: {ex.Message}");
        }
    }
}
```

Output

```
CSS Copy code

Driving a Car...
Riding a Bike...
Error: Invalid vehicle type
```

Advantages

Advantage	Description
Encapsulation of Object Creation	The creation logic is centralized and hidden from the client.
Loose Coupling	The client depends on abstractions, not concrete classes.
Improved Maintainability	Adding new object types only requires updating the factory, not client code.
Promotes Open/Closed Principle	New classes can be added with minimal code modification.

Disadvantages

Disadvantage	Description
Increased Complexity	Adds an extra layer (factory class) even for simple scenarios.
Modification Still Needed in Factory	Must update factory logic when new types are added (can be mitigated using Abstract Factory).
Less Transparent Code Flow	Developers must look into the factory to know which concrete classes are being created.

Alternatives / Related Patterns

Alternative Pattern	Description
Abstract Factory Pattern	Used when you have families of related objects to be created together.
Builder Pattern	Useful when object creation is complex and involves multiple steps.
Prototype Pattern	Used when you need to clone existing objects instead of creating new ones.
Dependency Injection (DI)	Delegates creation responsibility to a DI container instead of a factory class.

Unit Test Example (Simulated via Main Method)

Using a fake implementation for testing:

```
csharp  Copy code

public class FakeVehicle : IVehicle
{
    public void Drive()
    {
        Console.WriteLine("[TEST] Simulated driving operation");
    }
}

class Test
{
    static void Main()
    {
        IVehicle testVehicle = new FakeVehicle();
        testVehicle.Drive();
    }
}
```

Output

```
csharp  Copy code

[TEST] Simulated driving operation
```

Summary

The **Factory Pattern** is a **creational pattern** that hides the object creation logic and provides a centralized way to instantiate objects based on given input. It improves **loose coupling**, **maintainability**, and **code reusability**, making it ideal for applications that need to create multiple related object types dynamically — a common scenario in **.NET Core** and **microservice architectures**.

3) Singleton – A Creational Design Pattern:

🌸 Singleton Design Pattern – [Creational Pattern]

◆ Definition

The **Singleton Design Pattern** ensures that **only one instance** of a class is created throughout the application's lifetime, and it provides a **global point of access** to that instance.

🏠 Real-world Analogy

Think of a government passport office in your city:

- There's **only one official office** responsible for issuing passports.
- Whenever citizens need passport services, they all go to that **same office** instead of creating a new one.
- The office is like a **Singleton** — a single, shared instance accessible by everyone.

⚠️ Bad Example (Anti-Pattern) – Without Singleton

Here, multiple objects are created unnecessarily.

```
csharp
public class Logger
{
    public void Log(string message)
    {
        Console.WriteLine($"Log Entry: {message}");
    }
}

class Program
{
    static void Main(string[] args)
    {
        // Multiple instances (bad practice)
        var logger1 = new Logger();
        var logger2 = new Logger();

        logger1.Log("First Message");
        logger2.Log("Second Message");

        Console.WriteLine(object.ReferenceEquals(logger1, logger2)); // False
    }
}
```

✗ Output:

```
mathematica
Log Entry: First Message
Log Entry: Second Message
False
```

✗ Problem:

Each time you create a new `Logger`, it has its own instance.

This wastes memory and breaks consistency — especially for global things like logging, configuration, or caching.

✓ Good Example – Singleton Implementation

csharp

Copy code

```
public sealed class Logger
{
    // Private static instance
    private static readonly Logger _instance = new Logger();

    // Private constructor prevents direct instantiation
    private Logger()
    {
        Console.WriteLine("Logger Instance Created");
    }

    // Public property to access the single instance
    public static Logger Instance
    {
        get
        {
            return _instance;
        }
    }

    public void Log(string message)
    {
        Console.WriteLine($"Log Entry: {message}");
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        // Accessing the same instance multiple times
        var logger1 = Logger.Instance;
        var logger2 = Logger.Instance;

        logger1.Log("Application Started");
        logger2.Log("Application Running");

        Console.WriteLine(object.ReferenceEquals(logger1, logger2)); // True
    }
}
```

✓ Output:

mathematica

Copy code

```
Logger Instance Created
Log Entry: Application Started
Log Entry: Application Running
True
```

✓ Notice that:

- The constructor runs **only once**.
- `logger1` and `logger2` point to the same object.

🔗 How to Test in Main()

In the `Main()` method:

1. Get multiple references via `Logger.Instance`.
2. Log a few messages.
3. Use `ReferenceEquals(logger1, logger2)` to confirm both references are identical.

csharp

Copy code

```
var logger1 = Logger.Instance;
var logger2 = Logger.Instance;

logger1.Log("Hello from logger1");
logger2.Log("Hello from logger2");

Console.WriteLine($"Same instance? {object.ReferenceEquals(logger1, logger2)}");
```

✅ Output:

pgsql

Copy code

```
Logger Instance Created
Log Entry: Hello from logger1
Log Entry: Hello from logger2
Same instance? True
```

💡 Advantages

- ✅ Ensures only one instance throughout application lifetime.
- ✅ Provides a global, easy access point to shared resources (e.g., Logger, Config).
- ✅ Prevents conflicting states by maintaining a single instance.
- ✅ Saves memory and improves performance when only one instance is needed.

⚠️ Disadvantages

- ❌ Difficult to unit test because of the global state.
- ❌ Can lead to tight coupling if used everywhere directly.
- ❌ Can hide dependencies, making code less maintainable.
- ❌ Not suitable for multithreaded creation without care (though the above approach is thread-safe due to static initialization).

🔄 Alternatives to Singleton

1. **Dependency Injection (DI)**
 - Let the DI container manage the lifetime (e.g., `AddSingleton<T>()` in .NET Core).
 - Promotes loose coupling and testability.
2. **Static Class**
 - For pure utility methods (no instance state).
 - But lacks flexibility and interface implementation.
3. **Service Locator** (less preferred)
 - Central registry for retrieving services but often considered an anti-pattern.

Summary Table

Aspect	Description
Pattern Type	Creational
Purpose	Ensure a single, globally accessible instance
Common Use Cases	Logging, Configuration, Caching, Thread Pool, Database Connection
Best Practice	Prefer DI container-managed Singleton in .NET Core
Thread Safety	Achieved using static initialization or locks

4) Strategy - A Behavioral Design Pattern:

Strategy Pattern – [Behavioral Pattern]

◆ Definition

The **Strategy Design Pattern** defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime.

It allows the algorithm to vary **independently** from the clients that use it.

Real-world Analogy

Imagine a **payment system** in an e-commerce app:

- Customers can pay via **Credit Card**, **UPI**, or **PayPal**.
- The payment process (algorithm) varies, but the goal (payment) remains the same.
- Instead of writing multiple `if-else` conditions in your code, you can define each payment method as a **strategy**, and the system will decide which one to use **at runtime**.

✅ This makes it easy to **add or change payment methods** without modifying existing code.

⚠️ Bad Example (Anti-pattern) – Using if-else or switch

csharp

Copy code

```
public class PaymentProcessor
{
    public void MakePayment(string paymentType)
    {
        if (paymentType == "CreditCard")
        {
            Console.WriteLine("Paid using Credit Card.");
        }
        else if (paymentType == "UPI")
        {
            Console.WriteLine("Paid using UPI.");
        }
        else if (paymentType == "PayPal")
        {
            Console.WriteLine("Paid using PayPal.");
        }
        else
        {
            Console.WriteLine("Invalid payment method.");
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        PaymentProcessor processor = new PaymentProcessor();

        processor.MakePayment("CreditCard");
        processor.MakePayment("UPI");
        processor.MakePayment("PayPal");
    }
}
```

❌ Problems (Anti-pattern)

- Hard to extend — adding a new payment method means modifying `MakePayment()`.
- Violates **Open/Closed Principle** (class should be open for extension, closed for modification).
- Difficult to maintain and test each payment method independently.

✓ Good Example – Using Strategy Pattern

1. Step 1: Create a Strategy Interface

csharp

 Copy code

```
public interface IPaymentStrategy
{
    void Pay(decimal amount);
}
```

2. Step 2: Create Concrete Strategies

csharp

 Copy code

```
public class CreditCardPayment : IPaymentStrategy
{
    public void Pay(decimal amount)
    {
        Console.WriteLine($"Paid {amount:C} using Credit Card.");
    }
}

public class UPIPayment : IPaymentStrategy
{
    public void Pay(decimal amount)
    {
        Console.WriteLine($"Paid {amount:C} using UPI.");
    }
}

public class PayPalPayment : IPaymentStrategy
{
    public void Pay(decimal amount)
    {
        Console.WriteLine($"Paid {amount:C} using PayPal.");
    }
}
```

3. Step 3: Create the Context Class

csharp

 Copy code

```
public class PaymentContext
{
    private IPaymentStrategy _paymentStrategy;

    public void SetPaymentStrategy(IPaymentStrategy strategy)
    {
        _paymentStrategy = strategy;
    }

    public void MakePayment(decimal amount)
    {
        if (_paymentStrategy == null)
        {
            Console.WriteLine("Please select a payment method first.");
            return;
        }
        _paymentStrategy.Pay(amount);
    }
}
```

4. Step 4: Test in Main()

csharp

 Copy code

```
class Program
{
    static void Main(string[] args)
    {
        PaymentContext context = new PaymentContext();

        // Using Credit Card
        context.SetPaymentStrategy(new CreditCardPayment());
        context.MakePayment(2500);

        // Using UPI
        context.SetPaymentStrategy(new UPIPayment());
        context.MakePayment(800);

        // Using PayPal
        context.SetPaymentStrategy(new PayPalPayment());
        context.MakePayment(1200);
    }
}
```

Output:

cpp

 Copy code

```
Paid ₹2,500.00 using Credit Card.
Paid ₹800.00 using UPI.
Paid ₹1,200.00 using PayPal.
```

How to Test in Main Method

- Instantiate `PaymentContext`.
- Set a different payment strategy dynamically.
- Call `MakePayment()` to verify the correct behavior is executed.

For example:

csharp

 Copy code

```
context.SetPaymentStrategy(new CreditCardPayment());
context.MakePayment(1000);

context.SetPaymentStrategy(new UPIPayment());
context.MakePayment(500);
```

Advantages

- ✓ Adds flexibility — algorithms can be changed at runtime.
- ✓ Follows **Open/Closed Principle** — easily add new strategies without modifying existing code.
- ✓ Promotes cleaner and more modular code.
- ✓ Makes unit testing easier — test each strategy independently.
- ✓ Reduces `if-else` or `switch` clutter.

Disadvantages

- ✗ Slightly increases number of classes (one per strategy).
- ✗ Context and strategy must work closely, so overuse can add complexity.
- ✗ Selecting the correct strategy at runtime requires client knowledge.

Alternatives

Alternative	Description
State Pattern	Similar to Strategy but used when the behavior changes <i>based on object state</i> rather than user choice.
Command Pattern	Encapsulates an operation as an object — useful for undo/redo scenarios.
Dependency Injection	Can inject the desired strategy at runtime, reducing manual wiring.

Summary Table

Aspect	Description
Pattern Type	Behavioral
Purpose	Select and switch between multiple algorithms at runtime
Analogy	Choosing different payment methods in a shopping app
Best Use Case	When you have multiple algorithms for the same task
Key Principle	"Encapsulate what varies"

5) Observer – A Behavioral Design Pattern:

🌸 Observer Design Pattern – [Behavioral Pattern]

♦ Definition

The Observer Pattern defines a **one-to-many relationship** between objects — when one object (called the **Subject**) changes its state, all its **dependent objects (Observers)** are automatically notified and updated.

It helps achieve **loose coupling** between objects that need to stay in sync.

🏠 Real-world Analogy

Think of a YouTube Channel and Subscribers:

- When a YouTube Channel uploads a new video, all its **subscribers** get a notification.
- The channel doesn't need to know *who* the subscribers are — it just *notifies all*.
- Subscribers can **subscribe** or **unsubscribe** anytime.

👉 Here,

- YouTube Channel = Subject (Publisher)
- Subscribers = Observers

⚠ Bad Example (Anti-pattern) – Tight Coupling

csharp

Copy code

```
public class YouTubeChannel
{
    public void UploadVideo(string videoTitle)
    {
        Console.WriteLine($"Channel uploaded: {videoTitle}");
        NotifySubscribers(videoTitle);
    }

    private void NotifySubscribers(string videoTitle)
    {
        // Tightly coupled with specific subscribers
        var subscriber1 = new Subscriber("Alice");
        var subscriber2 = new Subscriber("Bob");

        subscriber1.Notify(videoTitle);
        subscriber2.Notify(videoTitle);
    }
}

public class Subscriber
{
    private string _name;
    public Subscriber(string name)
    {
        _name = name;
    }

    public void Notify(string videoTitle)
    {
        Console.WriteLine($"{_name} received notification: New video - {videoTitle}");
    }
}
```

```
class Program
{
    static void Main()
    {
        var channel = new YouTubeChannel();
        channel.UploadVideo("Design Patterns in .NET");
    }
}
```

✗ Problems

- The channel is hard-coded with subscriber instances.
- Adding/removing subscribers requires modifying `YouTubeChannel` class.
- Violates Open/Closed Principle — not easily extensible.

✔ Good Example – Using Observer Pattern Properly

1. Create Subject (Publisher) Interface

csharp

 Copy code

```
public interface ISubject
{
    void Attach(IObserver observer);
    void Detach(IObserver observer);
    void Notify(string videoTitle);
}
```

2. Create Observer Interface

csharp

 Copy code

```
public interface IObserver
{
    void Update(string videoTitle);
}
```

3. Implement Concrete Subject

csharp

 Copy code

```
public class YouTubeChannel : ISubject
{
    private readonly List<IObserver> _subscribers = new List<IObserver>();
    public string ChannelName { get; }

    public YouTubeChannel(string name)
    {
        ChannelName = name;
    }

    public void Attach(IObserver observer)
    {
        _subscribers.Add(observer);
        Console.WriteLine($"{(observer as Subscriber)?.Name} subscribed to {ChannelName}");
    }

    public void Detach(IObserver observer)
    {
        _subscribers.Remove(observer);
        Console.WriteLine($"{(observer as Subscriber)?.Name} unsubscribed from {ChannelName}");
    }

    public void Notify(string videoTitle)
    {
        foreach (var subscriber in _subscribers)
        {
            subscriber.Update(videoTitle);
        }
    }

    public void UploadVideo(string videoTitle)
    {
        Console.WriteLine($"\\nChannel '{ChannelName}' uploaded a new video: {videoTitle}");
        Notify(videoTitle);
    }
}
```

4. Implement Concrete Observer

csharp

 Copy code

```
public class Subscriber : IObservable
{
    public string Name { get; }

    public Subscriber(string name)
    {
        Name = name;
    }

    public void Update(string videoTitle)
    {
        Console.WriteLine($"{Name} received notification: New video uploaded - {videoTitle}");
    }
}
```

How to Test in Main Method

csharp

 Copy code

```
class Program
{
    static void Main()
    {
        // Create channel
        YouTubeChannel dotNetChannel = new YouTubeChannel("DotNetAcademy");

        // Create subscribers
        Subscriber alice = new Subscriber("Alice");
        Subscriber bob = new Subscriber("Bob");
        Subscriber charlie = new Subscriber("Charlie");

        // Subscribers attach to channel
        dotNetChannel.Attach(alice);
        dotNetChannel.Attach(bob);
        dotNetChannel.Attach(charlie);

        // Upload a video
        dotNetChannel.UploadVideo("Observer Pattern in .NET Core");

        // One subscriber unsubscribes
        dotNetChannel.Detach(charlie);

        // Upload another video
        dotNetChannel.UploadVideo("Strategy Pattern Explained");
    }
}
```

✓ Output

sql

 Copy code

```
Alice subscribed to DotNetAcademy
Bob subscribed to DotNetAcademy
Charlie subscribed to DotNetAcademy
```

```
Channel 'DotNetAcademy' uploaded a new video: Observer Pattern in .NET Core
Alice received notification: New video uploaded - Observer Pattern in .NET Core
Bob received notification: New video uploaded - Observer Pattern in .NET Core
Charlie received notification: New video uploaded - Observer Pattern in .NET Core
```

```
Charlie unsubscribed from DotNetAcademy
```

```
Channel 'DotNetAcademy' uploaded a new video: Strategy Pattern Explained
Alice received notification: New video uploaded - Strategy Pattern Explained
Bob received notification: New video uploaded - Strategy Pattern Explained
```

💡 Advantages

- ✓ Promotes loose coupling — Subject doesn't know observer details.
- ✓ Easy to extend — add or remove observers at runtime.
- ✓ Promotes event-driven programming.
- ✓ Reusable and maintainable design.

⚠ Disadvantages

- ✗ Notification order is not guaranteed.
- ✗ Too many observers can cause performance overhead.
- ✗ Difficult to debug complex chains of updates.
- ✗ Risk of memory leaks if observers aren't detached properly.

🔄 Alternatives

Alternative	Description
C# Events / Delegates	Built-in implementation of Observer pattern; cleaner and simpler for small cases.
Mediator Pattern	If many objects interact in complex ways, use a mediator to coordinate instead of direct notifications.
Reactive Extensions (Rx.NET)	Powerful reactive programming library — uses Observer pattern under the hood.

🌱 Summary Table

Aspect	Description
Pattern Type	Behavioral
Purpose	Notify multiple dependents automatically of state changes
Analogy	YouTube Channel notifying subscribers of new videos
Key Benefit	Loose coupling between publishers and subscribers
Best Use Case	Event systems, UI updates, message broadcasting

6)Repository – A Structural Design Pattern:

🌸 Repository Pattern – [Structural Pattern]

◆ Definition

The **Repository Pattern** acts as a mediator between the domain and data mapping layers, providing a collection-like interface for accessing domain objects.

It hides the complexities of data access logic (like Entity Framework, SQL, or API calls) behind a clean abstraction.

In simple words — it **decouples business logic from the data access layer**, allowing your application to change databases or data sources without affecting core logic.

🏠 Real-world Analogy

Imagine you run an **Online Bookstore**:

- You (the business layer) ask your **Book Repository** for books.
- The repository knows *how* to fetch them (from database, API, or cache).
- You don't care where the data comes from — you just need a list of books.

✅ The **Repository** acts like a **middleman librarian** who knows exactly where to find and return the books — you just request them.

⚠️ Bad Example (Anti-pattern) – Business Logic Directly Accesses Database

```
csharp

public class BookService
{
    private readonly string _connectionString = "Server=.;Database=BookStore;Trusted_Connection=True;";

    public List<string> GetBooks()
    {
        List<string> books = new List<string>();

        using (var connection = new SqlConnection(_connectionString))
        {
            connection.Open();
            SqlCommand cmd = new SqlCommand("SELECT Title FROM Books", connection);
            SqlDataReader reader = cmd.ExecuteReader();

            while (reader.Read())
            {
                books.Add(reader["Title"].ToString());
            }
        }
        return books;
    }
}
```

csharp

 Copy code

```
class Program
{
    static void Main()
    {
        BookService service = new BookService();
        var books = service.GetBooks();

        foreach (var book in books)
            Console.WriteLine(book);
    }
}
```

❌ Problems (Anti-pattern)

- Business logic is **tightly coupled** with data access (SQL, connection strings, etc.).
- Hard to **unit test** since it directly depends on a database.
- Violates **Single Responsibility Principle**.
- If you change the data source (say, to a Web API), you must rewrite logic everywhere.

✅ Good Example – Using Repository Pattern

1. Define the Model

csharp

 Copy code

```
public class Book
{
    public int Id { get; set; }
    public string Title { get; set; }
}
```

2. Define Repository Interface

csharp

 Copy code

```
public interface IBookRepository
{
    IEnumerable<Book> GetAll();
    Book GetById(int id);
    void Add(Book book);
    void Delete(int id);
}
```

3. Implement Repository

csharp

 Copy code

```
public class BookRepository : IBookRepository
{
    // Simulated database
    private readonly List<Book> _books = new List<Book>
    {
        new Book { Id = 1, Title = "Clean Code" },
        new Book { Id = 2, Title = "Design Patterns in .NET" },
        new Book { Id = 3, Title = "C# in Depth" }
    };

    public IEnumerable<Book> GetAll()
    {
        return _books;
    }

    public Book GetById(int id)
    {
        return _books.FirstOrDefault(b => b.Id == id);
    }

    public void Add(Book book)
    {
        _books.Add(book);
    }

    public void Delete(int id)
    {
        var book = _books.FirstOrDefault(b => b.Id == id);
        if (book != null)
            _books.Remove(book);
    }
}
```

4. Create a Service Layer that Uses Repository

csharp

 Copy code

```
public class BookService
{
    private readonly IBookRepository _repository;

    public BookService(IBookRepository repository)
    {
        _repository = repository;
    }

    public void DisplayAllBooks()
    {
        var books = _repository.GetAll();
        foreach (var book in books)
        {
            Console.WriteLine($"Book ID: {book.Id}, Title: {book.Title}");
        }
    }

    public void AddNewBook(string title)
    {
        var book = new Book { Id = new Random().Next(100, 999), Title = title };
        _repository.Add(book);
        Console.WriteLine($"Book '{title}' added successfully!");
    }
}
```

Testing in Main()

csharp

 Copy code

```
class Program
{
    static void Main()
    {
        IBookRepository repo = new BookRepository();
        BookService service = new BookService(repo);

        Console.WriteLine($"📖 Initial Books:");
        service.DisplayAllBooks();

        Console.WriteLine($"📝 Adding New Book:");
        service.AddNewBook("Microservices with .NET");

        Console.WriteLine($"📖 Updated Book List:");
        service.DisplayAllBooks();
    }
}
```

✓ Output

yaml

Copy code

```
Initial Books:
Book ID: 1, Title: Clean Code
Book ID: 2, Title: Design Patterns in .NET
Book ID: 3, Title: C# in Depth

+ Adding New Book:
Book 'Microservices with .NET' added successfully!

Updated Book List:
Book ID: 1, Title: Clean Code
Book ID: 2, Title: Design Patterns in .NET
Book ID: 3, Title: C# in Depth
Book ID: 847, Title: Microservices with .NET
```

💡 Advantages

- ✓ Separation of Concerns – Business logic is independent of data access logic.
- ✓ Easier to Unit Test – You can mock the repository during testing.
- ✓ Maintainability – Database-related changes affect only the repository layer.
- ✓ Reusability – Same repository can be reused across different services.
- ✓ Supports Multiple Data Sources – Can easily switch between EF Core, Dapper, or external APIs.

⚠ Disadvantages

- ✗ Adds extra abstraction (might be overkill for small apps).
- ✗ Sometimes leads to **duplicate query logic** between repository and EF DbContext.
- ✗ Can hide database-specific features (like LINQ-to-SQL advantages).
- ✗ Misuse (creating generic repositories for everything) can increase complexity.

🔄 Alternatives

Alternative	Description
Direct EF Core DbContext (No Repository)	Modern .NET apps sometimes use EF Core's DbContext directly in the service layer.
CQRS Pattern	Separates read and write operations into different models. Often used with Repository.
Unit of Work Pattern	Groups multiple repository operations under a single transaction.
Mediator Pattern	Used in modern CQRS implementations to coordinate commands/queries instead of repositories.

🌱 Summary Table

Aspect	Description
Pattern Type	Structural
Purpose	Abstracts data access logic and provides a clean interface for business logic
Analogy	Librarian fetching books for you
Best Use Case	CRUD-based enterprise apps using EF Core or similar ORM
Commonly Used With	Unit of Work, Dependency Injection
Example Libraries	EF Core, Dapper, MongoDB Driver

7)Unit of Work – A Structural Design Pattern:

🌱 Unit of Work Pattern – [Structural Pattern]

◆ Definition

The Unit of Work (UoW) Pattern is a design pattern that maintains a list of operations to be performed in a single transaction.

It:

- Coordinates changes across multiple repositories.
- Ensures all operations either **succeed together** or **fail together** (transactional consistency).
- Reduces repetitive database calls and improves maintainability.

In essence, Unit of Work acts as a transaction manager for repositories.

🏠 Real-world Analogy

Think of shopping in a supermarket:

- You pick multiple items from different sections (fruits, bakery, dairy).
- You take all items to the **checkout counter**.
- The cashier processes **all items in a single transaction**.
- Either all items are purchased successfully, or if payment fails, **none are purchased**.

✅ Here, the **checkout counter acts like a Unit of Work**, coordinating multiple item purchases as a **single unit of work**.

⚠️ Bad Example (Anti-pattern) – Repositories Called Independently

```
csharp Copy code

public class BookRepository
{
    private List<string> _books = new List<string>();
    public void AddBook(string title) => _books.Add(title);
    public void DisplayBooks() => _books.ForEach(Console.WriteLine);
}

public class AuthorRepository
{
    private List<string> _authors = new List<string>();
    public void AddAuthor(string name) => _authors.Add(name);
    public void DisplayAuthors() => _authors.ForEach(Console.WriteLine);
}

class Program
{
    static void Main()
    {
        var bookRepo = new BookRepository();
        var authorRepo = new AuthorRepository();

        // ❌ Adding book and author independently
        bookRepo.AddBook("Clean Code");
        authorRepo.AddAuthor("Robert Martin");

        // ❌ If one operation fails, the other succeeds
        // Example: Payment fails, but book is still added
        bookRepo.DisplayBooks();
        authorRepo.DisplayAuthors();
    }
}
```

❌ Problems

- Operations are **not transactional**.
- Adding multiple entities independently can lead to **data inconsistency**.
- Difficult to manage **commit/rollback** when multiple repositories are involved.

✓ Good Example – Using Unit of Work Pattern

1. Define Repositories

```
csharp Copy code

public interface IBookRepository
{
    void Add(Book book);
    void DisplayAll();
}

public interface IAuthorRepository
{
    void Add(Author author);
    void DisplayAll();
}

public class BookRepository : IBookRepository
{
    private readonly List<Book> _books = new();
    public void Add(Book book) => _books.Add(book);
    public void DisplayAll() => _books.ForEach(b => Console.WriteLine($"Book: {b.Title}"));
}

public class AuthorRepository : IAuthorRepository
{
    private readonly List<Author> _authors = new();
    public void Add(Author author) => _authors.Add(author);
    public void DisplayAll() => _authors.ForEach(a => Console.WriteLine($"Author: {a.Name}"));
}
```

2. Define Models

```
csharp Copy code

public class Book
{
    public string Title { get; set; }
}

public class Author
{
    public string Name { get; set; }
}
```


3. Implement Unit of Work

```
csharp Copy code

public interface IUnitOfWork
{
    IBookRepository Books { get; }
    IAuthorRepository Authors { get; }
    void Commit(); // Saves all changes
}

public class UnitOfWork : IUnitOfWork
{
    public IBookRepository Books { get; }
    public IAuthorRepository Authors { get; }

    public UnitOfWork()
    {
        Books = new BookRepository();
        Authors = new AuthorRepository();
    }

    public void Commit()
    {
        // In real scenario, this is where database transaction would be committed
        Console.WriteLine("All changes have been committed as a single transaction.\n");
    }
}
```

4. Test in Main Method

```
csharp Copy code

class Program
{
    static void Main()
    {
        IUnitOfWork unitOfWork = new UnitOfWork();

        // Add multiple entities
        unitOfWork.Books.Add(new Book { Title = "Clean Code" });
        unitOfWork.Books.Add(new Book { Title = "Design Patterns" });
        unitOfWork.Authors.Add(new Author { Name = "Robert Martin" });
        unitOfWork.Authors.Add(new Author { Name = "Gang of Four" });

        // Commit all changes together
        unitOfWork.Commit();

        // Display all
        unitOfWork.Books.DisplayAll();
        unitOfWork.Authors.DisplayAll();
    }
}
```

✓ Output

vbnet

Copy code

All changes have been committed as a single transaction.

Book: Clean Code

Book: Design Patterns

Author: Robert Martin

Author: Gang of Four

💡 Advantages

- ✓ Ensures transactional consistency across multiple repositories.
- ✓ Reduces duplicate database calls by batching operations.
- ✓ Decouples business logic from data access.
- ✓ Makes unit testing easier since repositories are centralized.
- ✓ Works seamlessly with Repository Pattern.

⚠ Disadvantages

- ✗ Adds extra abstraction — may be overkill for small applications.
- ✗ Slight complexity in implementation.
- ✗ Manual transaction management is required if not using EF Core or other ORMs with built-in transactions.

🔄 Alternatives

Alternative	Description	📄
Direct Repository Calls	Simpler but loses transactional consistency.	
TransactionScope in .NET	Handles transaction manually without UoW abstraction.	
CQRS Pattern	Separates read/write models, often used with Unit of Work for writes.	

🧩 Summary Table

Aspect	Description
Pattern Type	Structural
Purpose	Group multiple repository operations as a single transaction
Analogy	Supermarket checkout counter processing all items together
Best Use Case	Enterprise apps using multiple repositories/DB tables
Commonly Used With	Repository Pattern, EF Core, DI

8) CQRS (Command Query Responsibility Segregation) – An Architectural Design Pattern:

✿ CQRS – [Architectural Pattern]

◆ Definition

CQRS (Command Query Responsibility Segregation) is an architectural pattern that separates operations that read data (Queries) from operations that modify data (Commands).

- **Commands:** Modify state (Create, Update, Delete)
- **Queries:** Read state (Get, Fetch, List)

The key idea: Read and write responsibilities are handled separately, often with separate models, services, or even databases.

🏠 Real-world Analogy

Think of a banking system:

- **Queries:** Checking your account balance, viewing transaction history.
- **Commands:** Transferring money, depositing, or withdrawing funds.

You don't mix reading and writing logic together.

Separating them improves scalability, performance, and simplifies complex business rules.

⚠️ Bad Example (Anti-pattern) – Mixing Reads and Writes

```
csharp Copy code

public class BankAccountService
{
    private decimal _balance = 1000;

    public decimal GetBalance()
    {
        // Query
        return _balance;
    }

    public void Deposit(decimal amount)
    {
        // Command
        _balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= _balance)
            _balance -= amount;
        else
            throw new Exception("Insufficient balance");
    }

    public string GetAccountSummary()
    {
        // Query mixed with Logic
        return $"Balance: {_balance}";
    }
}
```

```
class Program
{
    static void Main()
    {
        var accountService = new BankAccountService();
        accountService.Deposit(500);
        Console.WriteLine(accountService.GetBalance());
        accountService.Withdraw(200);
        Console.WriteLine(accountService.GetAccountSummary());
    }
}
```

❌ Problems

- Read and write logic are tightly coupled.
- Hard to scale reads vs writes independently.
- Complex business logic makes the service class **hard to maintain**.

✓ Good Example – Using CQRS Pattern

1. Define Commands and Queries

```
csharp Copy code

// Command (modifies state)
public class DepositCommand
{
    public decimal Amount { get; set; }
}

// Command Handler
public class AccountCommandHandler
{
    private readonly BankAccount _account;
    public AccountCommandHandler(BankAccount account)
    {
        _account = account;
    }

    public void Handle(DepositCommand command)
    {
        _account.Balance += command.Amount;
        Console.WriteLine($"Deposited {command.Amount:C}. New Balance: {_account.Balance:C}");
    }

    public void Handle(WithdrawCommand command)
    {
        if (command.Amount <= _account.Balance)
        {
            _account.Balance -= command.Amount;
            Console.WriteLine($"Withdrew {command.Amount:C}. New Balance: {_account.Balance:C}");
        }
        else
        {
            Console.WriteLine("Insufficient balance!");
        }
    }
}

public class WithdrawCommand
{
    public decimal Amount { get; set; }
}
```

csharp

 Copy code

```
// Query (reads state)
public class GetBalanceQuery
{
}

public class AccountQueryHandler
{
    private readonly BankAccount _account;
    public AccountQueryHandler(BankAccount account)
    {
        _account = account;
    }

    public decimal Handle(GetBalanceQuery query)
    {
        return _account.Balance;
    }
}
```

2. Domain Model

csharp

 Copy code

```
public class BankAccount
{
    public decimal Balance { get; set; }
    public BankAccount(decimal initialBalance)
    {
        Balance = initialBalance;
    }
}
```

3. Test in Main Method

csharp

 Copy code

```
class Program
{
    static void Main()
    {
        BankAccount account = new BankAccount(1000);

        var commandHandler = new AccountCommandHandler(account);
        var queryHandler = new AccountQueryHandler(account);

        // Deposit money
        commandHandler.Handle(new DepositCommand { Amount = 500 });

        // Withdraw money
        commandHandler.Handle(new WithdrawCommand { Amount = 200 });

        // Check balance (Query)
        decimal balance = queryHandler.Handle(new GetBalanceQuery());
        Console.WriteLine($"Final Balance: {balance:C}");
    }
}
```

✓ Output

pgsql

Copy code

```
Deposited $500.00. New Balance: $1,500.00
Withdrew $200.00. New Balance: $1,300.00
Final Balance: $1,300.00
```

💡 Advantages

- ✓ Separation of concerns — read and write operations are separate.
- ✓ Scalability — can scale read-heavy and write-heavy workloads independently.
- ✓ Performance optimization — read and write models can be optimized differently.
- ✓ Simpler business logic — commands handle state changes, queries handle reads.
- ✓ Works well with Event Sourcing and Microservices.

⚠ Disadvantages

- ✗ Adds complexity — more classes and handlers to maintain.
- ✗ Slightly harder to understand for beginners.
- ✗ Requires careful handling of eventual consistency if using separate stores.
- ✗ Overkill for small/simple applications.

🔧 Alternatives

Alternative	Description
CRUD Service	Simpler approach where reads and writes are handled in one service (good for small apps).
Event Sourcing	Often used together with CQRS; stores events rather than current state.
Mediator Pattern	Can be used to implement command/query handling via a centralized mediator.

🌱 Summary Table

Aspect	Description
Pattern Type	Architectural / Behavioral
Purpose	Separate commands (writes) from queries (reads) for scalability and clarity
Analogy	Banking system: deposits/withdrawals vs checking balance
Best Use Case	Large systems with high read/write loads, microservices, event-driven architectures
Commonly Used With	Event Sourcing, Mediator Pattern, Microservices

9) Mediator – A Behavioral Design Pattern:

✿ Mediator Pattern – [Behavioral Pattern]

◆ Definition

The **Mediator Pattern** defines an object that encapsulates how a set of objects interact, promoting loose coupling.

- Instead of objects communicating directly with each other, they **communicate through a mediator**.
- Reduces dependencies between communicating objects.
- Makes the system easier to maintain and extend.

🏢 Real-world Analogy

Imagine a **air traffic control tower** at an airport:

- Multiple **airplanes (colleagues)** need to coordinate takeoff and landing.
- Instead of airplanes communicating with each other directly, they **communicate via the control tower (mediator)**.
- The tower ensures safe operations and coordinates timing.

✅ Here, the **Mediator** coordinates all communication between colleagues.

⚠️ Bad Example (Anti-pattern) – Tight Coupling

csharp

📄 Copy code

```
public class Airplane
{
    public string Name { get; set; }

    public Airplane(string name)
    {
        Name = name;
    }

    public void SendMessage(Airplane to, string message)
    {
        Console.WriteLine($"{Name} sends message to {to.Name}: {message}");
        to.ReceiveMessage(message);
    }

    public void ReceiveMessage(string message)
    {
        Console.WriteLine($"{Name} received: {message}");
    }
}

class Program
{
    static void Main()
    {
        Airplane plane1 = new Airplane("Flight A");
        Airplane plane2 = new Airplane("Flight B");

        // Planes communicate directly
        plane1.SendMessage(plane2, "Request landing clearance");
        plane2.SendMessage(plane1, "Acknowledged");
    }
}
```


❌ Problems

- Airplanes are **tightly coupled** — each needs to know about all other airplanes.
- Hard to scale — adding more airplanes increases code complexity.
- Violates **Single Responsibility Principle** — airplanes handle their own communication.

✅ Good Example – Using Mediator Pattern

1. Define Mediator Interface

```
csharp Copy code  
  
public interface IAirTrafficControl  
{  
    void SendMessage(string message, Airplane airplane);  
    void RegisterAirplane(Airplane airplane);  
}
```

2. Implement Concrete Mediator

```
csharp Copy code  
  
public class AirTrafficControl : IAirTrafficControl  
{  
    private List<Airplane> _airplanes = new List<Airplane>();  
  
    public void RegisterAirplane(Airplane airplane)  
    {  
        _airplanes.Add(airplane);  
    }  
  
    public void SendMessage(string message, Airplane sender)  
    {  
        foreach (var airplane in _airplanes)  
        {  
            if (airplane != sender)  
                airplane.ReceiveMessage(message);  
        }  
    }  
}
```

3. Define Colleague (Airplane)

csharp

 Copy code

```
public class Airplane
{
    private IAirTrafficControl _control;
    public string Name { get; set; }

    public Airplane(string name, IAirTrafficControl control)
    {
        Name = name;
        _control = control;
        _control.RegisterAirplane(this);
    }

    public void SendMessage(string message)
    {
        Console.WriteLine($"{Name} sends: {message}");
        _control.SendMessage(message, this);
    }

    public void ReceiveMessage(string message)
    {
        Console.WriteLine($"{Name} received: {message}");
    }
}
```

4. Test in Main Method

csharp

 Copy code

```
class Program
{
    static void Main()
    {
        IAirTrafficControl control = new AirTrafficControl();

        Airplane flightA = new Airplane("Flight A", control);
        Airplane flightB = new Airplane("Flight B", control);
        Airplane flightC = new Airplane("Flight C", control);

        flightA.SendMessage("Request landing clearance");
        flightB.SendMessage("Holding at 5000 ft");
        flightC.SendMessage("Ready for takeoff");
    }
}
```

✓ Output

yaml

Copy code

```
Flight A sends: Request landing clearance
Flight B received: Request landing clearance
Flight C received: Request landing clearance
Flight B sends: Holding at 5000 ft
Flight A received: Holding at 5000 ft
Flight C received: Holding at 5000 ft
Flight C sends: Ready for takeoff
Flight A received: Ready for takeoff
Flight B received: Ready for takeoff
```

💡 Advantages

- ✓ Reduces **direct dependencies** between objects (loose coupling).
- ✓ Simplifies object communication — easier to maintain and extend.
- ✓ Changes in one colleague **don't affect others**.
- ✓ Centralizes control logic in **one mediator**.

⚠️ Disadvantages

- ✗ Mediator can become a **god object** if it handles too much logic.
- ✗ Adds extra layer — may be overkill for simple systems.
- ✗ All colleagues are dependent on mediator; mediator failure may break communication.

🔄 Alternatives

Alternative	Description
Observer Pattern	For broadcast notifications without central control.
Event Aggregator	Similar to mediator but more loosely coupled and asynchronous.
Direct communication	Simple but leads to tight coupling (anti-pattern).

✿ Summary Table

Aspect	Description
Pattern Type	Behavioral
Purpose	Encapsulate object interactions and reduce dependencies
Analogy	Air traffic control tower coordinating multiple airplanes
Best Use Case	Complex systems with multiple interacting components
Commonly Used With	CQRS, Mediator libraries like MediatR in .NET Core

10) Decorator – A Structural Design Pattern:

✿ Decorator Pattern – [Structural Pattern]

◆ Definition

The Decorator Pattern allows behavior to be added to an individual object dynamically, without affecting the behavior of other objects from the same class.

- It's a flexible alternative to subclassing for extending functionality.
- The decorator wraps the original object and provides additional behavior.

🍷 Real-world Analogy

Think of a coffee shop:

- You order a plain coffee.
- You can add milk, sugar, chocolate, or whipped cream.
- Each addition decorates the coffee with extra features without creating a new coffee type for every combination.

✅ Here,

- Plain coffee = Core object
- Additions (milk, sugar, chocolate) = Decorators

⚠️ Bad Example (Anti-pattern) – Using Subclassing for Every Combination

```
csharp Copy code

public class Coffee
{
    public virtual string GetDescription() => "Plain Coffee";
    public virtual double GetCost() => 5.0;
}

public class CoffeeWithMilk : Coffee
{
    public override string GetDescription() => "Coffee with Milk";
    public override double GetCost() => 6.0;
}

public class CoffeeWithSugar : Coffee
{
    public override string GetDescription() => "Coffee with Sugar";
    public override double GetCost() => 5.5;
}

// Problem: For every combination (Milk + Sugar + Chocolate), need a new class
```

✗ Problems

- Explosion of subclasses for each combination.
- Hard to maintain and extend.
- Violates Open/Closed Principle (need to modify code to add new features).

✓ Good Example – Using Decorator Pattern

1. Define Component Interface

csharp

 Copy code

```
public interface ICoffee
{
    string GetDescription();
    double GetCost();
}
```

2. Implement Core Component

csharp

 Copy code

```
public class PlainCoffee : ICoffee
{
    public string GetDescription() => "Plain Coffee";
    public double GetCost() => 5.0;
}
```

3. Create Base Decorator

csharp

 Copy code

```
public abstract class CoffeeDecorator : ICoffee
{
    protected ICoffee _coffee;
    protected CoffeeDecorator(ICoffee coffee)
    {
        _coffee = coffee;
    }

    public virtual string GetDescription() => _coffee.GetDescription();
    public virtual double GetCost() => _coffee.GetCost();
}
```

4. Implement Concrete Decorators

csharp

 Copy code

```
public class MilkDecorator : CoffeeDecorator
{
    public MilkDecorator(ICoffee coffee) : base(coffee) { }

    public override string GetDescription() => _coffee.GetDescription() + ", Milk";
    public override double GetCost() => _coffee.GetCost() + 1.0;
}

public class SugarDecorator : CoffeeDecorator
{
    public SugarDecorator(ICoffee coffee) : base(coffee) { }

    public override string GetDescription() => _coffee.GetDescription() + ", Sugar";
    public override double GetCost() => _coffee.GetCost() + 0.5;
}

public class ChocolateDecorator : CoffeeDecorator
{
    public ChocolateDecorator(ICoffee coffee) : base(coffee) { }

    public override string GetDescription() => _coffee.GetDescription() + ", Chocolate";
    public override double GetCost() => _coffee.GetCost() + 2.0;
}
```

5. Test in Main Method

csharp

 Copy code

```
class Program
{
    static void Main()
    {
        ICoffee coffee = new PlainCoffee();
        Console.WriteLine($"{coffee.GetDescription()} - Cost: {coffee.GetCost():C}");

        // Add Milk
        coffee = new MilkDecorator(coffee);
        Console.WriteLine($"{coffee.GetDescription()} - Cost: {coffee.GetCost():C}");

        // Add Sugar
        coffee = new SugarDecorator(coffee);
        Console.WriteLine($"{coffee.GetDescription()} - Cost: {coffee.GetCost():C}");

        // Add Chocolate
        coffee = new ChocolateDecorator(coffee);
        Console.WriteLine($"{coffee.GetDescription()} - Cost: {coffee.GetCost():C}");
    }
}
```

✓ Output

pgsql

Copy code

```
Plain Coffee - Cost: $5.00
Plain Coffee, Milk - Cost: $6.00
Plain Coffee, Milk, Sugar - Cost: $6.50
Plain Coffee, Milk, Sugar, Chocolate - Cost: $8.50
```

💡 Advantages

- ✓ Adds functionality **dynamically** without altering original class.
- ✓ Promotes **Open/Closed Principle** — easy to extend behavior.
- ✓ Reduces the need for **multiple subclasses**.
- ✓ Decorators can be **combined in any order**.
- ✓ Transparent to client — client just uses the interface.

⚠ Disadvantages

- ✗ Can lead to many **small decorator classes**.
- ✗ Can make code **harder to read** if decorators are heavily nested.
- ✗ All decorators must implement the same interface.

🔄 Alternatives

Alternative	Description	📄
Subclassing	Less flexible; requires creating a class for each combination.	
Strategy Pattern	For changing behavior dynamically, but doesn't chain features like decorators.	
Extension Methods	Simple for adding methods, but can't store state or chain behaviors dynamically.	

🌱 Summary Table

Aspect	Description	📄
Pattern Type	Structural	
Purpose	Dynamically add behavior to objects without subclassing	
Analogy	Coffee with optional add-ons (milk, sugar, chocolate)	
Best Use Case	When multiple optional behaviors need to be combined flexibly	
Commonly Used With	Decorator can wrap any component interface; often combined with Dependency Injection	

11) Circuit Breaker – A Microservices Design Pattern:

✿ Circuit Breaker Pattern – [Microservices Pattern]

◆ Definition

The Circuit Breaker Pattern is a resilience pattern used in distributed systems (like microservices) to prevent repeated failures from overwhelming a service.

- It monitors for failures and stops sending requests to a failing service.
- Allows the system to fail fast instead of waiting for timeouts.
- After a timeout, it tests the service to see if it has recovered before sending more requests.

🔔 Real-world Analogy

Think of a circuit breaker in electricity:

- If too much current flows, the breaker trips to prevent damage.
- Once the issue is resolved, it can be reset to allow electricity flow again.

✅ Here,

- Service = Electrical appliance
- Circuit breaker = Safety switch
- Failures = Overcurrent

The pattern protects the system from cascading failures.

⚠️ Bad Example (Anti-pattern) – No Circuit Breaker

csharp

Copy code

```
public class PaymentService
{
    private int _failureCount = 0;

    public void ProcessPayment(string orderId)
    {
        _failureCount++;
        if (_failureCount % 3 != 0)
        {
            // Simulate failure
            throw new Exception("Payment service unavailable!");
        }

        Console.WriteLine($"Payment processed for order {orderId}");
    }
}

class Program
{
    static void Main()
    {
        var paymentService = new PaymentService();

        for (int i = 1; i <= 5; i++)
        {
            try
            {
                paymentService.ProcessPayment($"ORD{i}");
            }
            catch (Exception ex)
            {
                Console.WriteLine(ex.Message);
            }
        }
    }
}
```

❌ Problems

- System keeps sending requests to a failing service.
- Can overload the service.
- No mechanism to recover gracefully.

✓ Good Example – Circuit Breaker Implementation

1. Circuit Breaker Class

csharp

 Copy code

```
public class CircuitBreaker
{
    private int _failureThreshold;
    private int _failureCount;
    private bool _isOpen;
    private DateTime _lastAttempt;
    private TimeSpan _resetTimeout;

    public CircuitBreaker(int failureThreshold, TimeSpan resetTimeout)
    {
        _failureThreshold = failureThreshold;
        _resetTimeout = resetTimeout;
        _failureCount = 0;
        _isOpen = false;
    }

    public bool IsOpen => _isOpen;

    public void Execute(Action action)
    {
        if (_isOpen)
        {
            // Check if timeout has elapsed to attempt reset
            if (DateTime.Now - _lastAttempt > _resetTimeout)
            {
                _isOpen = false; // Attempt to reset
                Console.WriteLine("Circuit breaker reset. Trying service again...");
            }
            else
            {
                Console.WriteLine("Circuit is open. Skipping operation.");
                return;
            }
        }

        try
        {
            action();
            _failureCount = 0; // Reset failures on success
        }
        catch (Exception ex)
        {
            _failureCount++;
            Console.WriteLine($"Operation failed: {ex.Message}");

            if (_failureCount >= _failureThreshold)
            {
                _isOpen = true;
                _lastAttempt = DateTime.Now;
                Console.WriteLine("Circuit breaker opened!");
            }
        }
    }
}
```

2. Payment Service

csharp

 Copy code

```
public class PaymentService
{
    private int _callCount = 0;

    public void ProcessPayment(string orderId)
    {
        _callCount++;

        // Simulate failure for first 3 calls
        if (_callCount <= 3)
            throw new Exception("Payment service unavailable!");

        Console.WriteLine($"Payment processed for order {orderId}");
    }
}
```

3. Test in Main Method

csharp

```
class Program
{
    static void Main()
    {
        var paymentService = new PaymentService();
        var circuitBreaker = new CircuitBreaker(failureThreshold: 2, resetTimeout: TimeSpan.FromSeconds(5));

        for (int i = 1; i <= 6; i++)
        {
            Console.WriteLine($"Attempt {i}:");
            circuitBreaker.Execute(() => paymentService.ProcessPayment($"ORD{i}"));
            Thread.Sleep(1000); // Simulate time between calls
        }

        // Wait for reset
        Console.WriteLine($"Waiting for circuit breaker reset...");
        Thread.Sleep(6000);

        Console.WriteLine($"Attempt after reset:");
        circuitBreaker.Execute(() => paymentService.ProcessPayment("ORD7"));
    }
}
```

✓ Output

yaml

📄 Copy code

```
Attempt 1:  
Operation failed: Payment service unavailable!  
  
Attempt 2:  
Operation failed: Payment service unavailable!  
Circuit breaker opened!  
  
Attempt 3:  
Circuit is open. Skipping operation.  
  
Attempt 4:  
Circuit is open. Skipping operation.  
  
Attempt 5:  
Circuit is open. Skipping operation.  
  
Attempt 6:  
Circuit is open. Skipping operation.  
  
Waiting for circuit breaker reset...  
  
Attempt after reset:  
Circuit breaker reset. Trying service again...  
Payment processed for order ORD7
```

💡 Advantages

- ✓ Prevents **cascading failures** in microservices.
- ✓ **Fails fast**, avoiding resource exhaustion.
- ✓ Provides **graceful degradation** when services are unavailable.
- ✓ Allows **automatic recovery** after a timeout.

⚠️ Disadvantages

- ✗ Adds **complexity** in implementation.
- ✗ Needs careful tuning of **failure thresholds and timeout**.
- ✗ May **skip valid requests** if circuit is open too aggressively.
- ✗ Only works if failures can be detected reliably.

Alternatives / Complementary Patterns

Pattern	Description
Retry Pattern	Retry failed operations before tripping the circuit. Often combined with circuit breaker.
Bulkhead Pattern	Isolate failures to specific service partitions to prevent cascading failures.
Fallback Pattern	Provide an alternative response if service is unavailable.
Timeout Pattern	Limit how long a service call waits to prevent blocking.

Summary Table

Aspect	Description
Pattern Type	Microservices / Resilience
Purpose	Prevent repeated failures from overwhelming a service; fail fast
Analogy	Electrical circuit breaker protecting devices
Best Use Case	Distributed systems, microservices, unreliable external services
Commonly Used With	Retry, Timeout, Bulkhead, Fallback patterns

12) API Gateway – A Microservices Design Pattern:

API Gateway Pattern – [Microservices Pattern]

◆ Definition

The API Gateway Pattern is a design pattern in microservices where a **single entry point (gateway) handles all client requests**.

- The gateway routes requests to the appropriate microservice(s).
- It can also **handle cross-cutting concerns** like authentication, logging, rate limiting, caching, and response aggregation.
- Clients do **not call microservices directly**; they interact with the gateway.

🏠 Real-world Analogy

Think of a hotel concierge desk:

- Visitors request services: restaurant reservation, spa booking, taxi, or room service.
- Instead of going to each department individually, visitors go to the concierge (API Gateway).
- The concierge coordinates the requests with different departments and gives a single response.

✅ Here,

- Client = Visitor
- Microservices = Hotel departments
- API Gateway = Concierge desk

⚠️ Bad Example (Anti-pattern) – Direct Microservice Calls

```
csharp Copy code

public class UserService
{
    public string GetUser(int id) => $"User{id}";
}

public class OrderService
{
    public string GetOrders(int userId) => $"Orders for User{userId}";
}

class Program
{
    static void Main()
    {
        var userService = new UserService();
        var orderService = new OrderService();

        int userId = 1;
        // Client calls services directly
        string user = userService.GetUser(userId);
        string orders = orderService.GetOrders(userId);

        Console.WriteLine($"{user} has {orders}");
    }
}
```

❌ Problems

- Clients must know all microservices and endpoints.
- Changes in microservices break client code.
- Cannot handle cross-cutting concerns in one place.
- Hard to implement rate limiting, caching, or logging consistently.

✓ Good Example – Using API Gateway Pattern

1. Microservices (Simplified)

```
csharp Copy code

public class UserService
{
    public string GetUser(int id) => $"User{id}";
}

public class OrderService
{
    public string GetOrders(int userId) => $"Orders for User{userId}";
}

public class PaymentService
{
    public string GetPayments(int userId) => $"Payments for User{userId}";
}
```

2. API Gateway

```
csharp Copy code

public class ApiGateway
{
    private readonly UserService _userService;
    private readonly OrderService _orderService;
    private readonly PaymentService _paymentService;

    public ApiGateway()
    {
        _userService = new UserService();
        _orderService = new OrderService();
        _paymentService = new PaymentService();
    }

    public string GetUserDashboard(int userId)
    {
        // Aggregates responses from multiple microservices
        var user = _userService.GetUser(userId);
        var orders = _orderService.GetOrders(userId);
        var payments = _paymentService.GetPayments(userId);

        return $"{user}\n{orders}\n{payments}";
    }
}
```

3. Test in Main Method

csharp

 Copy code

```
class Program
{
    static void Main()
    {
        var apiGateway = new ApiGateway();
        string dashboard = apiGateway.GetUserDashboard(1);

        Console.WriteLine("User Dashboard:");
        Console.WriteLine(dashboard);
    }
}
```

Output

rust

 Copy code

```
User Dashboard:
User1
Orders for User1
Payments for User1
```

Advantages

-  Single entry point for clients.
-  Reduces **client coupling** to microservices.
-  Can handle **cross-cutting concerns** centrally: authentication, logging, caching, rate limiting.
-  Supports **response aggregation** from multiple microservices.
-  Makes microservices easier to evolve **without breaking clients**.

Disadvantages

-  Can become a **bottleneck** if not scaled properly.
-  Adds **another layer** to maintain.
-  Gateway failure may affect all clients.
-  Can introduce **latency** due to request aggregation.

Alternatives

Alternative	Description	
Direct Client-to-Microservice Calls	Simple, but tightly couples clients to services (anti-pattern).	
Backend-for-Frontend (BFF)	Creates specialized gateways for different client types (e.g., mobile, web).	
Service Mesh	Handles cross-cutting concerns at the network layer rather than in a gateway.	

🌸 Summary Table

Aspect	Description
Pattern Type	Microservices / Architectural
Purpose	Provide a single entry point for clients and centralize cross-cutting concerns
Analogy	Hotel concierge desk coordinating multiple departments
Best Use Case	Large microservices architecture, multiple client types
Commonly Used With	Circuit Breaker, Rate Limiting, Authentication, Logging, Caching

13) Saga - A Microservices Transaction Design Pattern:

🌸 Saga Pattern – [Microservices Transaction Pattern]

♦ Definition

The **Saga Pattern** is a design pattern for managing **distributed transactions in microservices** without using a global database transaction.

- A **saga** is a sequence of **local transactions** across multiple services.
- Each local transaction updates the database and **publishes an event**.
- If a step fails, **compensating transactions** are triggered to undo previous changes.
- Ensures **eventual consistency** in a distributed system.

🏠 Real-world Analogy

Think of booking a travel package:

1. Book flight ✅
 2. Book hotel ✅
 3. Book rental car ❌ (fails)
- Instead of leaving a partial booking, the system **cancels previous bookings** (flight and hotel).
 - Each booking is a **local transaction**, and cancellations are **compensating transactions**.
- ✅ Here, the saga **ensures eventual consistency** without a single distributed transaction.

⚠️ Bad Example (Anti-pattern) – Direct Calls Without Compensation

```
csharp Copy code

public class BookingService
{
    public void BookFlight() => Console.WriteLine("Flight booked");
    public void BookHotel() => Console.WriteLine("Hotel booked");
    public void BookCar() => throw new Exception("Car booking failed!");
}

class Program
{
    static void Main()
    {
        var service = new BookingService();
        try
        {
            service.BookFlight();
            service.BookHotel();
            service.BookCar(); // Fails
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error: {ex.Message}");
        }

        // ❌ Flight and hotel remain booked, inconsistent state
    }
}
```

❌ Problems

- Partial failure leaves **inconsistent state**.
- No rollback mechanism.
- Distributed transaction not handled — violates **data consistency**.

✓ Good Example – Using Saga Pattern

1. Define Local Transactions

```
csharp Copy code

public class BookingService
{
    public void BookFlight() => Console.WriteLine("Flight booked");
    public void CancelFlight() => Console.WriteLine("Flight canceled");

    public void BookHotel() => Console.WriteLine("Hotel booked");
    public void CancelHotel() => Console.WriteLine("Hotel canceled");

    public void BookCar()
    {
        Console.WriteLine("Booking car...");
        throw new Exception("Car booking failed!");
    }
    public void CancelCar() => Console.WriteLine("Car booking canceled");
}
```

2. Implement Saga Coordinator

```
csharp Copy code

public class TravelBookingSaga
{
    private readonly BookingService _service;

    public TravelBookingSaga(BookingService service)
    {
        _service = service;
    }

    public void Execute()
    {
        try
        {
            _service.BookFlight();
            _service.BookHotel();
            _service.BookCar();
            Console.WriteLine("All bookings successful!");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error: {ex.Message}");
            Console.WriteLine("Triggering compensating transactions...");

            // Rollback previously successful transactions
            _service.CancelHotel();
            _service.CancelFlight();
        }
    }
}
```

3. Test in Main Method

csharp

 Copy code

```
class Program
{
    static void Main()
    {
        var bookingService = new BookingService();
        var saga = new TravelBookingSaga(bookingService);

        saga.Execute();
    }
}
```

Output

javascript

 Copy code

```
Flight booked
Hotel booked
Booking car...
Error: Car booking failed!
Triggering compensating transactions...
Hotel canceled
Flight canceled
```

Advantages

- ✓ Ensures eventual consistency in distributed systems.
- ✓ Avoids blocking distributed transactions.
- ✓ Each microservice handles its own local transaction independently.
- ✓ Supports failure recovery via compensating actions.

Disadvantages

- ✗ More complex than single transactions.
- ✗ Requires careful design of compensating actions.
- ✗ Harder to maintain eventual consistency guarantees in very complex workflows.
- ✗ Can lead to long-running processes.

Alternatives

Pattern	Description	
Two-Phase Commit (2PC)	Global distributed transaction, blocks resources, less scalable.	
Event Sourcing	Track all events, can rebuild state and handle compensation.	
Outbox Pattern	Ensure reliable event delivery between services.	

🌿 Summary Table

Aspect	Description
Pattern Type	Microservices / Transaction Management
Purpose	Manage distributed transactions with local transactions and compensations
Analogy	Travel booking: flight, hotel, car; cancel previous if a step fails
Best Use Case	Microservices with multiple dependent operations across services
Commonly Used With	Event-driven architecture, Message brokers, Outbox pattern

14) Event Sourcing – A Microservices Design Pattern:

🌿 Event Sourcing Pattern – [Microservices Pattern]

◆ Definition

Event Sourcing is a pattern where **state changes of a system are stored as a sequence of events**, rather than storing the current state directly.

- Each event represents a **change** in the system (e.g., "OrderPlaced", "PaymentProcessed").
- The current state is **reconstructed by replaying events**.
- Works well with **microservices** for auditing, CQRS, and consistency.

📖 Real-world Analogy

Think of a **bank account ledger**:

- Instead of storing only the **current balance**, the bank records **all transactions**: deposits, withdrawals, transfers.
- The **current balance** is calculated by summing up all transactions.
- You can **replay transactions** to reconstruct the balance at any point in time.

✅ Here,

- **Transactions = Events**
- **Account balance = Current state derived from events**

⚠️ Bad Example (Anti-pattern) – Only Store Current State

csharp

Copy code

```
public class BankAccount
{
    public decimal Balance { get; private set; }

    public void Deposit(decimal amount)
    {
        Balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= Balance)
            Balance -= amount;
        else
            throw new Exception("Insufficient balance");
    }
}

class Program
{
    static void Main()
    {
        var account = new BankAccount();
        account.Deposit(100);
        account.Withdraw(50);

        Console.WriteLine($"Current Balance: {account.Balance}");
    }
}
```

✖ Problems

- Cannot audit previous transactions easily.
- Hard to reconstruct state at a given point in time.
- No event log for analytics or debugging.

✓ Good Example – Using Event Sourcing

1. Define Events

csharp

 Copy code

```
public abstract class Event
{
    public DateTime Timestamp { get; set; } = DateTime.Now;
}

public class DepositedEvent : Event
{
    public decimal Amount { get; set; }
}

public class WithdrawnEvent : Event
{
    public decimal Amount { get; set; }
}
```

2. Event Store

csharp

 Copy code

```
public class EventStore
{
    private List<Event> _events = new List<Event>();

    public void Save(Event e)
    {
        _events.Add(e);
    }

    public IEnumerable<Event> GetEvents() => _events;
}
```

3. Bank Account with Event Sourcing

csharp

 Copy code

```
public class BankAccount
{
    private readonly EventStore _eventStore;
    public decimal Balance { get; private set; }

    public BankAccount(EventStore eventStore)
    {
        _eventStore = eventStore;
    }

    public void Deposit(decimal amount)
    {
        var e = new DepositedEvent { Amount = amount };
        _eventStore.Save(e);
        Apply(e);
    }

    public void Withdraw(decimal amount)
    {
        if (amount > Balance)
            throw new Exception("Insufficient balance");

        var e = new WithdrawnEvent { Amount = amount };
        _eventStore.Save(e);
        Apply(e);
    }
}
```

```
private void Apply(Event e)
{
    switch (e)
    {
        case DepositedEvent de:
            Balance += de.Amount;
            break;
        case WithdrawnEvent we:
            Balance -= we.Amount;
            break;
    }
}

public void Replay()
{
    Balance = 0;
    foreach (var e in _eventStore.GetEvents())
    {
        Apply(e);
    }
}
```


4. Test in Main Method

csharp

 Copy code

```
class Program
{
    static void Main()
    {
        var store = new EventStore();
        var account = new BankAccount(store);

        account.Deposit(100);
        account.Withdraw(30);
        account.Deposit(50);

        Console.WriteLine($"Current Balance: {account.Balance}");

        // Replay events to reconstruct state
        account.Replay();
        Console.WriteLine($"Balance after replay: {account.Balance}");

        Console.WriteLine("\nEvent History:");
        foreach (var e in store.GetEvents())
        {
            Console.WriteLine($"{e.Timestamp}: {e.GetType().Name}");
        }
    }
}
```

✓ Output

yaml

 Copy code

```
Current Balance: 120
Balance after replay: 120

Event History:
10/11/2025 14:20: DepositedEvent
10/11/2025 14:20: WithdrawnEvent
10/11/2025 14:20: DepositedEvent
```

💡 Advantages

- ✓ Full audit trail of all state changes.
- ✓ Can reconstruct state at any point in time.
- ✓ Works well with CQRS — events can drive separate read models.
- ✓ Supports debugging, analytics, and replay.

⚠ Disadvantages

- ✗ More complex than storing current state.
- ✗ Event versioning can be **challenging** as the system evolves.
- ✗ Event replay may become **slow** if the event log grows large.
- ✗ Requires **robust event storage and management**.

📦 Alternatives

Pattern	Description	📄
Traditional CRUD	Store only current state; simpler but lacks audit and replay capability.	
Snapshot + Event Sourcing	Store periodic snapshots to speed up replay.	
Change Data Capture (CDC)	Track changes in the database for auditing/analytics.	

🌱 Summary Table

Aspect	Description
Pattern Type	Microservices / Data Management
Purpose	Store all state changes as events to reconstruct current state
Analogy	Bank ledger with transaction history
Best Use Case	Microservices, auditing, CQRS, analytics
Commonly Used With	CQRS, Saga Pattern, Event-driven architecture

15) Proxy – A Structural Design Pattern:

🌱 Proxy Pattern – [Structural Pattern]

◆ Definition

The **Proxy Pattern** provides a **surrogate** or **placeholder** for another object to control access to it.

- A proxy can **add extra functionality** like caching, logging, security, or lazy initialization.
- The client interacts with the **proxy** instead of the real object.
- The real object is only accessed **when necessary**.

Real-world Analogy

Think of a security guard at a building entrance:

- Visitors cannot directly access the building (real object).
- The **guard** (proxy) checks credentials, logs access, or blocks unauthorized entry.
- Once approved, the visitor can access the building.

✅ Here,

- Security guard = Proxy
- Building = Real object
- Visitor = Client

Bad Example (Anti-pattern) – Direct Access

csharp

 Copy code

```
public class BankAccount
{
    public string Owner { get; set; }
    public decimal Balance { get; private set; }

    public BankAccount(string owner)
    {
        Owner = owner;
        Balance = 1000;
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= Balance)
        {
            Balance -= amount;
            Console.WriteLine($"{amount} withdrawn. Remaining: {Balance}");
        }
        else
        {
            Console.WriteLine("Insufficient funds!");
        }
    }
}

class Program
{
    static void Main()
    {
        var account = new BankAccount("John");
        account.Withdraw(500);
        account.Withdraw(700); // No access control or logging
    }
}
```

❌ Problems

- No control over who can access the object.
- No logging or security features.
- Client interacts directly with the sensitive object.

✅ Good Example – Using Proxy Pattern

1. Define Subject Interface

csharp

Copy code

```
public interface IBankAccount
{
    void Withdraw(decimal amount);
}
```

2. Implement Real Subject

csharp

Copy code

```
public class BankAccount : IBankAccount
{
    public string Owner { get; private set; }
    public decimal Balance { get; private set; }

    public BankAccount(string owner)
    {
        Owner = owner;
        Balance = 1000;
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= Balance)
        {
            Balance -= amount;
            Console.WriteLine($"{amount} withdrawn. Remaining: {Balance}");
        }
        else
        {
            Console.WriteLine("Insufficient funds!");
        }
    }
}
```

3. Implement Proxy

csharp

 Copy code

```
public class BankAccountProxy : IBankAccount
{
    private readonly BankAccount _realAccount;
    private readonly string _currentUser;

    public BankAccountProxy(BankAccount account, string currentUser)
    {
        _realAccount = account;
        _currentUser = currentUser;
    }

    public void Withdraw(decimal amount)
    {
        if (_currentUser == _realAccount.Owner)
        {
            Console.WriteLine($"[Proxy] Logging: {_currentUser} is withdrawing {amount}");
            _realAccount.Withdraw(amount);
        }
        else
        {
            Console.WriteLine($"[Proxy] Access denied for {_currentUser}");
        }
    }
}
```

4. Test in Main Method

csharp

 Copy code

```
class Program
{
    static void Main()
    {
        var account = new BankAccount("John");

        // Access via proxy
        IBankAccount proxy1 = new BankAccountProxy(account, "John");
        proxy1.Withdraw(500); // Allowed

        IBankAccount proxy2 = new BankAccountProxy(account, "Alice");
        proxy2.Withdraw(200); // Denied
    }
}
```

✓ Output

csharp

 Copy code

```
[Proxy] Logging: John is withdrawing 500
500 withdrawn. Remaining: 500
[Proxy] Access denied for Alice
```

💡 Advantages

- ✓ Controls access to the real object (security, validation).
- ✓ Can add logging, caching, lazy loading, or authorization without modifying real object.
- ✓ Reduces direct coupling between client and real object.
- ✓ Can improve performance by delaying expensive object creation.

⚠️ Disadvantages

- ✗ Adds extra layer of complexity.
- ✗ Can increase memory footprint if many proxies are created.
- ✗ May introduce slight latency due to additional method calls.

🔄 Alternatives

Pattern	Description
Decorator Pattern	Adds new behavior dynamically but doesn't control access.
Adapter Pattern	Changes interface of an existing object; does not add access control.
Service Layer / Facade	Provides centralized access but usually for simplifying APIs rather than security.

🌱 Summary Table

Aspect	Description
Pattern Type	Structural
Purpose	Control access, add extra functionality, or lazy initialization for an object
Analogy	Security guard controlling access to a building
Best Use Case	Sensitive objects, remote services, expensive object creation
Commonly Used With	Lazy loading, logging, caching, security

Design Patterns Summary

Category	Pattern	Common Usage
 Creational	Dependency Injection	Manage service lifetimes & decouple dependencies
	Factory	Dynamic object creation based on input or type
	Singleton	Global shared service or resource
 Structural	Repository + Unit of Work	Abstract data access & transaction management
	Decorator	Add behavior dynamically without changing core class
	Proxy	Lazy loading, access control, or API wrappers
 Behavioral	Strategy	Switch algorithms or behavior at runtime
	Observer	Event-driven communication between components
	Mediator	Decouple interactions between multiple objects
 Architectural	CQRS	Separate read and write operations for scalability
 Microservices	Circuit Breaker	Fault tolerance & fail-fast behavior
	Saga	Distributed transaction handling across services
	API Gateway	Unified entry point & routing for microservices
	Event Sourcing	Reconstruct state by storing events instead of current state

Top Use Cases for Design Patterns in .NET Core & Microservices applications:

1. Dependency Injection (Creational)

1. Injecting **services into controllers** (ASP.NET Core MVC/Web API) for decoupling business logic.
2. Managing **service lifetimes** (Scoped, Singleton, Transient) to optimize memory and performance.
3. Decoupling **data access layers** from business services for easier maintenance.
4. Injecting **cross-cutting concerns** like logging, caching, or configuration providers.
5. Facilitating **unit testing** by mocking dependencies without changing code.

2. Factory (Creational)

1. Creating **different database contexts** dynamically (e.g., SQL Server, PostgreSQL).
2. Producing **DTOs or domain objects** based on runtime input.
3. Selecting **payment processors** (PayPal, Stripe, Card) based on user choice.
4. Generating **microservice clients** dynamically depending on environment or version.
5. Instantiating **algorithm or policy objects** at runtime (e.g., shipping or discount strategies).

3. Singleton (Creational)

1. Central **configuration manager** shared across the application.
2. Global **caching service** for frequently accessed data.
3. Shared **logging provider** to maintain consistent logs across modules.
4. Database **connection pool manager** to reduce connection overhead.
5. Global **feature toggle manager** for enabling/disabling features in production.

4. Strategy (Behavioral)

1. Dynamically switching **pricing or discount algorithms** in e-commerce.
2. Selecting **search/sort algorithms** based on user requirements.
3. Choosing **payment strategies** at runtime (credit card, PayPal, wallet).
4. Configuring **logging strategies** (console, file, cloud).
5. Implementing **validation strategies** for different types of requests.

5. Observer (Behavioral)

1. Triggering **real-time notifications** (email, SMS, push) when events occur.
2. Broadcasting events to **microservices** in event-driven architecture.
3. Updating **UI components** automatically when the underlying model changes (SignalR).
4. Recording **audit logs** when domain events happen.
5. Synchronizing **cache or read models** when data changes in the database.

6. Repository (Structural)

1. Abstracting **CRUD operations** on entities for clean code.
2. Switching between **data sources** (SQL, NoSQL) without changing business logic.
3. Encapsulating **complex LINQ queries** for reusable operations.
4. Enabling **unit testing** by mocking repository interfaces.
5. Implementing **generic repository patterns** to handle multiple entities consistently.

7. Unit of Work (Structural)

1. Managing **transactional consistency** across multiple repositories.
2. Committing **all changes in one atomic transaction**.
3. Rolling back **partial updates** if a step fails.
4. Coordinating **multiple writes** in microservices.
5. Reducing **dependency complexity** by combining repositories under one unit.

8. CQRS (Architectural)

1. Separating **read and write operations** for scalability and performance.
2. Optimizing **read-heavy microservices** with denormalized views.
3. Handling **complex business rules** in the write model without affecting reads.
4. Supporting **event-driven read model updates**.
5. Enabling **audit trails** and history tracking of commands.

9. Mediator (Behavioral)

1. Decoupling request and response handling (e.g., using MediatR).
2. Centralizing commands and queries in microservices.
3. Handling domain events without tight coupling between components.
4. Reducing direct dependencies between services and modules.
5. Implementing pipeline behaviors (logging, validation, retry, authorization).

10. Decorator (Structural)

1. Adding logging dynamically to services without modifying core logic.
2. Adding caching layer to repository queries for performance.
3. Implementing authorization or security checks dynamically.
4. Extending payment or notification services with additional behaviors.
5. Combining multiple enhancements on the same service dynamically.

11. Circuit Breaker (Microservices)

1. Preventing cascading failures in dependent microservices.
2. Failing fast when third-party APIs are unavailable.
3. Protecting services from overload during high traffic spikes.
4. Integrating with Polly to add retries, fallbacks, and timeout handling.
5. Ensuring graceful degradation of user experience when failures occur.

12. API Gateway (Microservices)

1. Providing a single entry point for all client requests.
2. Routing requests to appropriate microservices.
3. Centralizing authentication, authorization, and rate limiting.
4. Aggregating multiple service responses into one payload.
5. Handling versioning and protocol translation for clients.

13. Saga (Microservices Transaction)

1. Coordinating distributed transactions across multiple microservices.
2. Managing multi-step workflows (e.g., order, payment, shipping).
3. Ensuring eventual consistency when a step fails.
4. Triggering compensating transactions to rollback partial work.
5. Implementing booking or reservation systems across services.

14. Event Sourcing (Microservices)

1. Storing all state changes as events instead of only current state.
2. Reconstructing aggregate state by replaying events.
3. Maintaining full audit logs and history.
4. Feeding read models in CQRS for optimized queries.
5. Supporting analytics and reporting by replaying historical events.

15. Proxy (Structural)

1. Implementing **lazy initialization** of expensive objects.
 2. Adding **access control** for sensitive resources or services.
 3. Adding **caching** for remote API calls.
 4. Logging and monitoring **method calls or service usage**.
 5. Wrapping **external APIs or microservices** to add extra behavior transparently.
-

Let's connect and share ideas on LinkedIn:

Sudam Amboji – [linkedin.com/in/sudam-amboji-2986859a](https://www.linkedin.com/in/sudam-amboji-2986859a)